

Logistics Optimization for Delivery Routes – Flipkart

Task 1: Data Cleaning & Preparation

1. Identify and delete duplicate Order_ID records. **No duplicates were found**
2. Replace null Traffic_Delay_Min with the average delay for that route. **No NULLS were found**
3. Convert all date columns into YYYY-MM-DD format using SQL functions. **Done**
4. Ensure that no Actual_Delivery_Date is before Order_Date (flag such records). **No records were found**

```
CREATE DATABASE FLIPKART;

USE FLIPKART;

SELECT * FROM ORDERS;

SELECT * FROM ROUTES;

SELECT * FROM WAREHOUSES;

SELECT * FROM DELIVERYAGENTS;

SELECT * FROM SHIPMENT_TRACKING;

-- Q1: Identify and delete duplicate Order_ID records

SELECT ORDER_ID
FROM
    ORDERS
GROUP BY ORDER_ID
HAVING COUNT(*) >1;

-- Q2: Replace null Traffic_Delay_Min with the average delay for that route

SELECT *
FROM
    ROUTES
WHERE
    TRAFFIC_DELAY_MIN IS NULL;
```

```

-- Using a Temporary Table as a Workaround

CREATE TEMPORARY TABLE TEMP_AVG_TRAFFIC_DELAY AS
SELECT AVG(TRAFFIC_DELAY_MIN) AS AVG_TRAFFIC_DELAY_MIN
FROM ROUTES;

-- Update NULL values if any with the average value of TRAFFIC_DELAY_MIN
UPDATE ROUTES
JOIN TEMP_AVG_TRAFFIC_DELAY
SET ROUTES.TRAFFIC_DELAY_MIN = TEMP_AVG_TRAFFIC_DELAY.AVG_TRAFFIC_DELAY_MIN
WHERE
TRAFFIC_DELAY_MIN IS NULL;

-- Q3: Convert all date columns into YYYY-MM-DD format using SQL functions

ALTER TABLE ORDERS ADD (ORDER_DT DATE);
UPDATE ORDERS SET ORDER_DT = STR_TO_DATE(ORDER_DATE , '%d-%m-%Y');
ALTER TABLE ORDERS DROP ORDER_DATE;
ALTER TABLE ORDERS RENAME COLUMN ORDER_DT TO ORDER_DATE;

ALTER TABLE ORDERS ADD (EXPECTED_DELIVERY_DT DATE);
UPDATE ORDERS SET EXPECTED_DELIVERY_DT = STR_TO_DATE(EXPECTED_DELIVERY_DATE
, '%d-%m-%Y');
ALTER TABLE ORDERS DROP EXPECTED_DELIVERY_DATE;
ALTER TABLE ORDERS RENAME COLUMN EXPECTED_DELIVERY_DT TO
EXPECTED_DELIVERY_DATE;

ALTER TABLE ORDERS ADD (ACTUAL_DELIVERY_DT DATE);
UPDATE ORDERS SET ACTUAL_DELIVERY_DT = STR_TO_DATE(ACTUAL_DELIVERY_DATE ,
'%d-%m-%Y');
ALTER TABLE ORDERS DROP ACTUAL_DELIVERY_DATE;
ALTER TABLE ORDERS RENAME COLUMN ACTUAL_DELIVERY_DT TO ACTUAL_DELIVERY_DATE;

```

-- Q4: Ensure that no Actual_Delivery_Date is before Order_Date (flag such records)

```
SELECT *  
  
FROM  
  
    ORDERS  
  
WHERE  
  
    ACTUAL_DELIVERY_DATE < ORDER_DATE;
```

Task 2: Delivery Delay Analysis

1. Calculate delivery delay (in days) for each order.
2. Find Top 10 delayed routes based on average delay days.
3. Use window functions to rank all orders by delay within each warehouse.

-- Q1: Calculate delivery delay (in days) for each order

```
SELECT  
  
    ORDER_ID, ROUND(SUM(DELAY_MINUTES)/1440,2) AS DELAY_IN_DAYS  
  
FROM  
  
    SHIPMENT_TRACKING  
  
GROUP BY ORDER_ID;
```

-- Q2: Find Top 10 delayed routes based on average delay days

```
SELECT  
  
    R.ROUTE_ID, R.START_LOCATION, R.END_LOCATION,  
    TOP_10_DELAYED_ROUTES.DELAY_IN_DAYS  
  
FROM  
  
    ROUTES R  
  
INNER JOIN  
  
    (  
  
        SELECT  
  
            O.ROUTE_ID, ROUND(SUM(ST.DELAY_MINUTES)/1440,2) AS  
            DELAY_IN_DAYS  
  
        FROM  
  
            ORDERS O
```

```

        INNER JOIN

            SHIPMENT_TRACKING ST

        ON O.ORDER_ID = ST.ORDER_ID

        GROUP BY O.ROUTE_ID

        ORDER BY DELAY_IN_DAYS DESC

        LIMIT 10) TOP_10_DELAYED_ROUTES

ON R.ROUTE_ID = TOP_10_DELAYED_ROUTES.ROUTE_ID

ORDER BY TOP_10_DELAYED_ROUTES.DELAY_IN_DAYS DESC;

```

-- Q3: Use window functions to rank all orders by delay within each warehouse

```

SELECT

    O.WAREHOUSE_ID, O.ORDER_ID, ORDER_DELAYS.DELAY_IN_DAYS, RANK()
OVER(PARTITION BY O.WAREHOUSE_ID ORDER BY ORDER_DELAYS.DELAY_IN_DAYS DESC)
AS RANKING

FROM

    ORDERS O

INNER JOIN

    (

        SELECT

            O.ORDER_ID, ROUND(SUM(ST.DELAY_MINUTES)/1440,2) AS
DELAY_IN_DAYS

        FROM

            SHIPMENT_TRACKING ST

        INNER JOIN

            ORDERS O

        ON ST.ORDER_ID = O.ORDER_ID

        GROUP BY O.ORDER_ID

    ) ORDER_DELAYS

ON O.ORDER_ID = ORDER_DELAYS.ORDER_ID;

```

Task 3: Route Optimization Insights

1. For each route, calculate:
 - Average delivery time (in days).
 - Average traffic delay.
 - Distance-to-time efficiency ratio: $\text{Distance_KM} / \text{Average_Travel_Time_Min}$.
2. Identify 3 routes with the worst efficiency ratio.
3. Find routes with >20% delayed shipments.
4. Recommend potential routes for optimization.

-- Q1.1: For each route calculate: Average delivery time (in days)

```
SELECT
    ROUTE_ID, ROUND(AVG(ACTUAL_DELIVERY_DATE - ORDER_DATE), 2) AS
    AVERAGE_DELIVERY_IN_DAYS
FROM
    ORDERS
GROUP BY ROUTE_ID
ORDER BY ROUTE_ID;
```

-- Q1.2: Average traffic delay

```
SELECT
    O.ROUTE_ID, ROUND(AVG(DELAY_MINUTES)/1440, 2) AS
    AVERAGE_TRAFFIC_DELAY_IN_DAYS
FROM
    SHIPMENT_TRACKING ST
INNER JOIN
    ORDERS O
ON ST.ORDER_ID = O.ORDER_ID
WHERE
    ST.DELAY_REASON = 'Traffic'
GROUP BY O.ROUTE_ID;
```

-- Q1.3: Distance-to-time efficiency ratio: Distance_KM / Average_Travel_Time_Min

```
SELECT

    *, ROUND((DISTANCE_KM/ AVERAGE_TRAVEL_TIME_MIN), 2) AS
    EFFICIENCY_RATIO

FROM

    ROUTES;
```

-- Q2: Identify 3 routes with the worst efficiency ratio

```
SELECT

    *, ROUND((DISTANCE_KM/ AVERAGE_TRAVEL_TIME_MIN), 2) AS
    EFFICIENCY_RATIO

FROM ROUTES

ORDER BY EFFICIENCY_RATIO ASC

LIMIT 3;
```

-- Q3: Find routes with >20% delayed shipments

```
SELECT

    T1.ROUTE_ID, T1.NO_OF_SHIPMENTS, T2.NO_OF_DELAYED_SHIPMENTS,
    ROUND((T1.NO_OF_SHIPMENTS * 0.2)) AS 20PERCENT_SHIPMENTS

FROM

    (SELECT

        O.ROUTE_ID, COUNT(ST.TRACKING_ID) AS NO_OF_SHIPMENTS

    FROM

        ORDERS O

    INNER JOIN

        SHIPMENT_TRACKING ST

    ON O.ORDER_ID = ST.ORDER_ID

    GROUP BY O.ROUTE_ID

    ORDER BY O.ROUTE_ID ASC) T1

    INNER JOIN

    (SELECT

        O.ROUTE_ID, COUNT(ST.TRACKING_ID) AS NO_OF_DELAYED_SHIPMENTS

    FROM

        ORDERS O

    INNER JOIN

        SHIPMENT_TRACKING ST
```

```

ON O.ORDER_ID = ST.ORDER_ID
WHERE DELAY_MINUTES > 0
GROUP BY O.ROUTE_ID
ORDER BY O.ROUTE_ID ASC) T2
ON T1.ROUTE_ID = T2.ROUTE_ID;

```

Task 4: Warehouse Performance

1. Find the top 3 warehouses with the highest average processing time.
2. Calculate total vs. delayed shipments for each warehouse.
3. Use CTEs to find bottleneck warehouses where processing time > global average.
4. Rank warehouses based on on-time delivery percentage.

```

-- Q1 : Find the top 3 warehouses with the highest average processing time.
SELECT *
FROM
    WAREHOUSES
ORDER BY AVERAGE_PROCESSING_TIME_MIN DESC
LIMIT 3;

-- Q2: Calculate total vs. delayed shipments for each warehouse
-- Total shipments
SELECT
    WH.WAREHOUSE_ID, WH.WAREHOUSE_NAME,
    TOTAL_SHIPMENTS_WH.NUM_OF_SHIPMENTS,
    DELAYED_SHIPMENTS_WH.NUM_OF_DELAYED_SHIPMENTS
FROM
    WAREHOUSES WH
LEFT JOIN
    (
        SELECT
            W.WAREHOUSE_ID, COUNT(ST.TRACKING_ID) NUM_OF_SHIPMENTS
        FROM
            WAREHOUSES W
INNER JOIN

```

```

        ORDERS O
    ON O.WAREHOUSE_ID = W.WAREHOUSE_ID
    INNER JOIN
        SHIPMENT_TRACKING ST
    ON ST.ORDER_ID = O.ORDER_ID
    GROUP BY WAREHOUSE_ID
) TOTAL_SHIPMENTS_WH
ON WH.WAREHOUSE_ID = TOTAL_SHIPMENTS_WH.WAREHOUSE_ID
LEFT JOIN
(
    SELECT
        W.WAREHOUSE_ID, COUNT(ST.TRACKING_ID) NUM_OF_DELAYED_SHIPMENTS
    FROM
        WAREHOUSES W
    INNER JOIN
        ORDERS O
    ON O.WAREHOUSE_ID = W.WAREHOUSE_ID
    INNER JOIN
        SHIPMENT_TRACKING ST
    ON ST.ORDER_ID = O.ORDER_ID
    WHERE
        ST.DELAY_REASON <> 'None'
    GROUP BY WAREHOUSE_ID
    ORDER BY W.WAREHOUSE_ID ASC
) DELAYED_SHIPMENTS_WH
ON
TOTAL_SHIPMENTS_WH.WAREHOUSE_ID = DELAYED_SHIPMENTS_WH.WAREHOUSE_ID
ORDER BY WH.WAREHOUSE_ID ASC;

```


-- Q3: Use CTEs to find bottleneck warehouses where processing time > global average

```
WITH CTE_WH_PROCESSING_TIME AS
(
SELECT
    WH1.WAREHOUSE_ID, WH1.WAREHOUSE_NAME, WH1.Average_Processing_Time_Min
AS AVG_PROCESSING_TIME, WH2.GLOBAL_AVERAGE
FROM
    WAREHOUSES WH1
CROSS JOIN
    (SELECT ROUND(AVG(Average_Processing_Time_Min),2) AS GLOBAL_AVERAGE
FROM WAREHOUSES) WH2
)
SELECT *
FROM
    CTE_WH_PROCESSING_TIME
WHERE
    AVG_PROCESSING_TIME > GLOBAL_AVERAGE;
```

-- Q4: Rank warehouses based on on-time delivery percentage

```
SELECT
    WH.WAREHOUSE_ID, WH.AVG_ON_TIME_DELIVERY_PECENTAGE, RANK() OVER(ORDER
BY WH.AVG_ON_TIME_DELIVERY_PECENTAGE DESC) AS RANKING
FROM
    (
SELECT
    W.WAREHOUSE_ID, ROUND(AVG(DA.ON_TIME_DELIVERY_PERCENTAGE),2) AS
AVG_ON_TIME_DELIVERY_PECENTAGE
FROM
    WAREHOUSES W
INNER JOIN
    ORDERS O
ON W.WAREHOUSE_ID = O.WAREHOUSE_ID
INNER JOIN
    DELIVERYAGENTS DA
```

```
ON O.AGENT_ID = DA.AGENT_ID

GROUP BY W.WAREHOUSE_ID

ORDER BY W.WAREHOUSE_ID ASC

) WH;
```

Task 5: Delivery Agent Performance

1. Rank agents (per route) by on-time delivery percentage.
2. Find agents with on-time % < 80%.
3. Compare average speed of top 5 vs bottom 5 agents using subqueries.
4. Suggest training or workload balancing strategies for low performers.

```
-- Q1: Rank agents (per route) by on-time delivery percentage

SELECT

    *, RANK() OVER(ORDER BY ON_TIME_DELIVERY_PERCENTAGE DESC) AS
ON_TIME_DELIVERY_RANK

FROM

    DELIVERYAGENTS;

-- Q2: Find agents with on-time % < 80%

SELECT *

FROM

    DELIVERYAGENTS

WHERE

    ON_TIME_DELIVERY_PERCENTAGE < 80;

-- Q3: Compare average speed of top 5 vs bottom 5 agents using subqueries

(SELECT

    *, TOP5'

FROM

    DELIVERYAGENTS

ORDER BY AVG_SPEED_KMPH DESC

LIMIT 5)
```

```
UNION

(SELECT
    *, 'BOTTOM5'
FROM
    DELIVERYAGENTS
ORDER BY AVG_SPEED_KMPH ASC
LIMIT 5);
```

Task 6: Shipment Tracking Analytics

1. For each order, list the last checkpoint and time.
2. Find the most common delay reasons (excluding None).
3. Identify orders with >2 delayed checkpoints

```
-- Q1: For each order, list the last checkpoint and time

WITH CTE_CHECKPOINT_TIME AS
(
SELECT
    ORDER_ID, CHECKPOINT_TIME, RANK() OVER(PARTITION BY ORDER_ID ORDER BY
CHECKPOINT_TIME DESC) AS CHECKPOINT_TIME_RANK
FROM
    SHIPMENT_TRACKING
ORDER BY ORDER_ID
)
SELECT
    ORDER_ID, CHECKPOINT_TIME AS LAST_CHECKPOINT_TIME
FROM
    CTE_CHECKPOINT_TIME
WHERE
    CHECKPOINT_TIME_RANK = 1
ORDER BY ORDER_ID;
```

-- Q2: Find the most common delay reasons (excluding None)

```
SELECT
    DELAY_REASON, COUNT(*)
FROM
    SHIPMENT_TRACKING
WHERE
    DELAY_REASON <> 'None'
GROUP BY DELAY_REASON
ORDER BY COUNT(*) DESC;
```

-- Q3: Identify orders with >2 delayed checkpoints

```
SELECT
    ORDER_ID, COUNT(CHECKPOINT) AS NUM_OF_DELAYED_CHECKPOINTS
FROM
    SHIPMENT_TRACKING
WHERE
    DELAY_REASON <> 'None'
GROUP BY ORDER_ID
HAVING NUM_OF_DELAYED_CHECKPOINTS > 2
ORDER BY ORDER_ID;
```

Task 7: Advanced KPI Reporting

1. Calculate KPIs using SQL queries: Average Delivery Delay per Region (Start Location)
2. On-Time Delivery % = (Total On-Time Deliveries / Total Deliveries) * 100
3. Average Traffic Delay per Route.

-- Q1: Average Delivery Delay per Region (Start_Location)

```
SELECT

    R.START_LOCATION, ROUND(AVG(ST.DELAY_MINUTES),2) AS
AVERAGE_DELIVERY_DELAY_MINUTES

FROM

    ROUTES R

INNER JOIN

    ORDERS O

ON O.ROUTE_ID = R.ROUTE_ID

INNER JOIN

    SHIPMENT_TRACKING ST

ON ST.ORDER_ID = O.ORDER_ID

GROUP BY R.START_LOCATION;
```

-- Q2: On-Time Delivery % = (Total On-Time Deliveries / Total Deliveries) * 100

```
SELECT

    ROUND((TOTAL_ONTIME_DELIVERIES/TOTAL_DELIVERIES),2) * 100 AS
ONTIME_DELIVERY_PERCENTAGE, TOTAL_ONTIME_DELIVERIES, TOTAL_DELIVERIES

FROM

    (

SELECT

    COUNT(*) TOTAL_ONTIME_DELIVERIES

FROM

    ORDERS

WHERE

    ACTUAL_DELIVERY_DATE = EXPECTED_DELIVERY_DATE

    ) A

CROSS JOIN

    (

SELECT COUNT(*) AS TOTAL_DELIVERIES FROM ORDERS

    ) B;
```

-- Q3: Average Traffic Delay per Route

```
SELECT
    O.ROUTE_ID, ROUND(AVG(ST.DELAY_MINUTES), 2) AS
    AVERAGE_TRAFFIC_DELAY_IN_MINUTES
FROM
    ORDERS O
INNER JOIN
    SHIPMENT_TRACKING ST
ON O.ORDER_ID = ST.ORDER_ID
WHERE
    ST.DELAY_REASON = 'Traffic'
GROUP BY O.ROUTE_ID
ORDER BY O.ROUTE_ID;
```